3.3 Storage-related sources

Next, we will take a closer look at storage-related sources. These sources range from relational databases to file systems. In general, since few people have to write their own storage systems, in this category, it's mostly about identifying what signals are available, in what form, and for what period of time (retention).

3.3.1 Relational databases and NoSQL data stores

Any useful real-world application typically has some state. In the context of cloud-native systems, there are several different relational database management systems (RDBMS), such as PostgreSQL and MySQL, as well as NoSQL datastores (e.g., DynamoDB, Redis, and MongoDB) used in microservices, depending on the requirements, such as strong consistency, scalability, and access patterns.

Regardless of the type of database, from a user point of view, the most important question to answer is how fast a query is processed. There are several techniques databases employ to speed up query processing, from using indexes to caches. In general, you want to treat the database as a black box, unless your role is specifically focused on this part, such as a database admin. To learn a little more about relevant metrics on a high level, check out the article, "Top 6 Database Performance Metrics to Monitor in Enterprise Applications," by Omed Habib (http://mng.bz/vn24).

RELATIONAL DATABASE MANAGEMENT SYSTEMS

The most common signal types found in the context of an RDBMS are logs and metrics, with traces also being increasingly available. Capturing the database query metrics, such as most frequent or slowest query as well as database performance (e.g., caches and sessions), provides the basis for operations.

High Performance MySQL, Fourth Edition, by Silvia Botros and Jeremy Tinley (http://mng.bz/4Dyj) is a great resource to learn about the kinds of signals MYSQL provides. It can also be used as a basis to generalize the relevant signals to other RDBMSs, such as Maria DB (https://mariadb.com/kb/en/server-monitoring-logs/). Let's take a look at a concrete example: in figure 3.5, you can see the Amazon Redshift performance monitoring, using Grafana as a frontend.

You can collect relevant database metrics yourself (e.g., the popular RDBMS PostgreSQL):

- Using Prometheus with the PostgreSQL Server Exporter (http://mng.bz/QP1v)
- Using a dedicated tool, like pgMonitor (https://github.com/CrunchyData/pgmonitor)

Another option to handle RDBMS performance metrics is to outsource this task to a vendor, be that an observability-focused vendor, like Dynatrace (http://mng.bz/XNQa), Datadog (https://www.datadoghq.com/blog/postgresql-monitoring/), or New Relic (http://mng.bz/yQ6d). Alternatively, you can use what your cloud provider of choice offers, with AWS (see figure 3.5), Azure SQL Database (http://mng.bz/MBAB), or Cloud SQL Insights (http://mng.bz/a1Ym).